Step 4 — La funzione obiettivo (loss), con esempi numerici

Obiettivo

Scegliere che cosa ottimizzare. La regola guida: **si addestra esattamente ciò che si legge in inferenza**, cioè il softmax sui logit delle lettere delle opzioni nella posizione dopo `Answer:`. Tutto il resto del vocabolario non viene toccato direttamente.

1. Che cosa si legge

Il modello, nella posizione dell'ultimo token del prompt, produce un logit per ogni token del vocabolario (~260.000 per Winnow, ~150.000 per JPT). Si prendono **solo** i logit delle lettere:

```
z = [z_A, z_B]                    # 2 numeri presi da ~260.000
p = softmax(z / T)                # T = 1 (Winnow), 1,087 (JPT)
```

Esempio Winnow (A = falso, B = vero), coppia con etichetta "supportato" (target B):

Momento	z_A	z_B	P(vero)
Prima di un passo	2,1	0,3	1 / (1 + e^(2,1 − 0,3)) = **0,142**
Dopo qualche passo	0,5	1,5	1 / (1 + e^(0,5 − 1,5)) = **0,731**

2. Perché non un SFT classico

L'SFT classico calcola la cross-entropy del token giusto **su tutto il vocabolario**. Situazione: P(A) = 0,05, P(B) = 0,40, e 0,55 sparso su altri token (a capo, "B.", "true", ...). Target B.

Loss	Calcolo	Valore
SFT classico	−ln 0,40	**0,92**
CE ristretta alle lettere	P(B fra A e B) = 0,40 / 0,45 = 0,889 → −ln 0,889	**0,12**

La decisione che si legge in inferenza è già giusta e sicura (0,889). L'SFT classico continua comunque a spingere forte, per spostare su B la massa degli altri token. Su migliaia di esempi:

- modifica parti del modello che in inferenza non si leggono;
- allarga la distanza fra z_B e *tutti* gli altri logit, compreso z_A: probabilità **sovra-confidenti**, soglia ottimale che scivola verso 0 o 1, calibrazione persa.

L'SFT classico resta un'approssimazione usabile con qualsiasi framework senza codice custom: l'esperimento P3 misura quanto si perde.

3. Cross-entropy o Brier sulle lettere

Con p = probabilità della lettera giusta:

Loss	Formula (2 opzioni)	Esempio p = 0,142	Gradiente rispetto a d = z_giusta − z_sbagliata
Cross-entropy (CE)	−ln p	1,95	−(1 − p) = **−0,86**
Brier (somma sulle opzioni, come in JPT)	Σ (p_k − y_k)² = 2(1 − p)²	1,47	−4p(1 − p)² = **−0,42**

La differenza vera è negli **errori molto sicuri**. Esempio con etichetta "vero" ma P(vero) = 0,01:

- CE: gradiente −0,99, la spinta massima. Giusto se l'etichetta è corretta; dannoso se è sbagliata (e nei dati sintetici, nei NEI di FEVER accoppiati in automatico o in alcune modifiche di SummEdits, etichette sbagliate ci sono).

- Brier: gradiente $-4 \cdot 0{,}01 \cdot 0{,}98 = -0{,}04$, quasi nessuna spinta. Robusto al rumore, ma lento a correggere errori veri e sicuri.

Situazione	Scelta
Default, dati puliti	**CE** (converge più in fretta)
Molti dati generati o rumorosi	Brier, oppure CE + Brier
JPT	Brier è coerente con il suo training originale: da provare per primo nell'esperimento P2

4. Loss di replay: KL verso il modello di partenza

Per il replay (02c) il target è la distribuzione del modello di partenza (teacher), su 2 o più opzioni. Si usa la divergenza di Kullback-Leibler:

```
KL(teacher ‖ studente) = Σ_k p_teacher[k] · ln(p_teacher[k] / p_studente[k])
```

Esempio, domanda `action` a 4 opzioni:

Opzione	Teacher	Studente dopo alcuni passi
continue	0,22	0,15
human_review	0,48	0,62
observe	0,27	0,20
stop	0,03	0,03

$\mathrm{KL} = 0{,}22 \cdot \ln(0{,}22/0{,}15) + 0{,}48 \cdot \ln(0{,}48/0{,}62) + 0{,}27 \cdot \ln(0{,}27/0{,}20) + 0 = 0{,}084 - 0{,}123 + 0{,}081 = \mathbf{0{,}042}$.

Se lo studente risponde come il teacher la KL è 0: il replay penalizza solo l'allontanamento dal comportamento originale, non l'"errore" rispetto a una verità.

5. Termine KL anche sul fact checking (opzionale)

Aggiungere `β · KL(teacher ‖ studente)` anche agli esempi di fact checking, con β piccolo (0,05–0,1). Effetto: il modello corregge gli errori, ma dove il modello di partenza era già giusto non si sposta inutilmente. Utile se la calibrazione peggiora o se l'oblio resta alto anche con il replay.

6. La loss totale

```
L = media sugli esempi di fact checking [ CE (o Brier) ]
  + λ · media sugli esempi di replay [ KL(teacher ‖ studente) ]
  + β · media sugli esempi di fact checking [ KL(teacher ‖ studente) ]        (opzionale)
  + μ · loss contrastiva sulle coppie                                          (opzionale, §8)
```

Valori iniziali: $\lambda = 1$, $\beta = 0$, $\mu = 0$. Si accendono β e μ solo negli esperimenti dedicati.

7. Codice di riferimento (per `finetune/train.py`, da scrivere)

```python
import torch
import torch.nn.functional as F

def letter_logits(logits, answer_pos, letter_ids, temperature):
    """logits [B, L, V] -> logit delle lettere [B, n_opzioni], già divisi per la temperatura."""
    rows = torch.arange(logits.size(0), device=logits.device)
    return logits[rows, answer_pos][:, letter_ids] / temperature

def decision_loss(z, target, kind="ce"):
    """z: [B, n] logit delle lettere; target: [B, n] one-hot (fact checking) o distribuzione
     ↪  (replay)."""
```

```
    if kind == "ce":
        return -(target * z.log_softmax(-1)).sum(-1).mean()
    if kind == "brier":
        return ((z.softmax(-1) - target) ** 2).sum(-1).mean()
    if kind == "kl":                                    # KL(teacher || studente)
        return F.kl_div(z.log_softmax(-1), target, reduction="batchmean")
    raise ValueError(kind)
```

Note pratiche:

- Si calcolano i logit solo nella posizione `answer_pos` (non su tutta la sequenza): risparmia memoria, perché la proiezione sul vocabolario intero per ogni posizione è la parte più costosa.
- Esempi con numero di opzioni diverso (2 per il fact checking, 3–5 per `choice` del replay) vanno raggruppati per numero di opzioni o gestiti con padding e maschera.

8. Estensione: loss contrastive

CE e Brier giudicano ogni esempio da solo. Un fact checker deve soprattutto **distinguere casi quasi identici**. Una loss contrastiva lavora su coppie.

8.1 Ranking logistico su coppie minime (consigliata)

Margine di ciascun esempio: `s = z_vero − z_falso` (per Winnow `z_B − z_A`; per JPT `(z_A − z_B) / 1,087`). Data una coppia (positivo, negativo) che differisce di poco:

`L_pair = −ln σ(s_pos − s_neg)`

Esempio, documento "The meeting was moved to Tuesday at 10:00.":

Claim	Etichetta	s	P(vero)	CE da sola
"The meeting is on Tuesday."	1	1,2	0,77	0,26
"The meeting is on Wednesday."	0	0,9	0,71	1,24

Il modello separa appena i due claim: s_pos − s_neg = 0,3, L_pair = $-\ln \sigma(0{,}3) =$ **0,55**. La loss spinge ad aumentare la differenza, cioè a guardare proprio la parola che cambia. Con una differenza di 3 la loss scende a 0,05.

Coppie disponibili: VitaminC (stesso `doc_id` = stesso caso), coppie generate con `remove_evidence` e `contradict_claim` (`meta.pair_id`), SummEdits (originale contro modificato; serve salvare `seed_summary` in `build_pairs.py`).

8.2 Coerenza fra varianti della domanda

Stessa coppia, due varianti della domanda: le probabilità dovrebbero coincidere. `L_cons = ½ [KL(p1 ‖ p2) + KL(p2 ‖ p1)]`. Esempio: 0,80 contro 0,60 → circa **0,10**.

8.3 Contrastiva sulle rappresentazioni (sconsigliata)

InfoNCE o SupCon sugli stati nascosti servono per addestrare encoder di embedding. Qui la decisione si legge dai logit delle lettere: vincolare le rappresentazioni non garantisce probabilità migliori, e servono batch grandi, difficili con documenti lunghi.

8.4 Aspetti pratici

- I due membri di una coppia devono stare nello stesso passo di ottimizzazione (stesso micro-batch, o due forward consecutivi prima del passo).
- Usarla su una parte degli esempi (20–30%), con $\mu \in \{0{,}1; 0{,}3; 0{,}5\}$: esperimenti X1–X4 in 07_esperimenti.md.

Criteri per passare allo step successivo

- Funzioni di loss scritte e testate su casi numerici noti (quelli di questo file).
- Scelta iniziale fissata: CE + replay KL ($\lambda = 1$).